



DLMCSPSE01 – Project Software Engineering

Pre-Code Unit Test Case Generator

Phase 2 – Development & Reflection

Raksha Rajkumar Kademani
4252827





Outline

- The Problem
 - Proposed Solution
 - Target Users
 - Six Core Features
 - Technology Stack
 - Database Design
 - Architecture – System Component Overview
 - Architecture – Data Flow
 - Agile / Scrum Sprint Plan
 - Functional Requirements
 - Non-Functional Requirements
 - Risk Register and Mitigation
 - References
- 

The Problem

Manual TDD Is Slowing Developers Down

Test-Driven Development requires unit test cases **before** writing any implementation code.

Today, developers create Excel spreadsheets by hand a process that is:

Time-Consuming

Repeated manual effort per function

Error-Prone

Human mistakes in edge case coverage

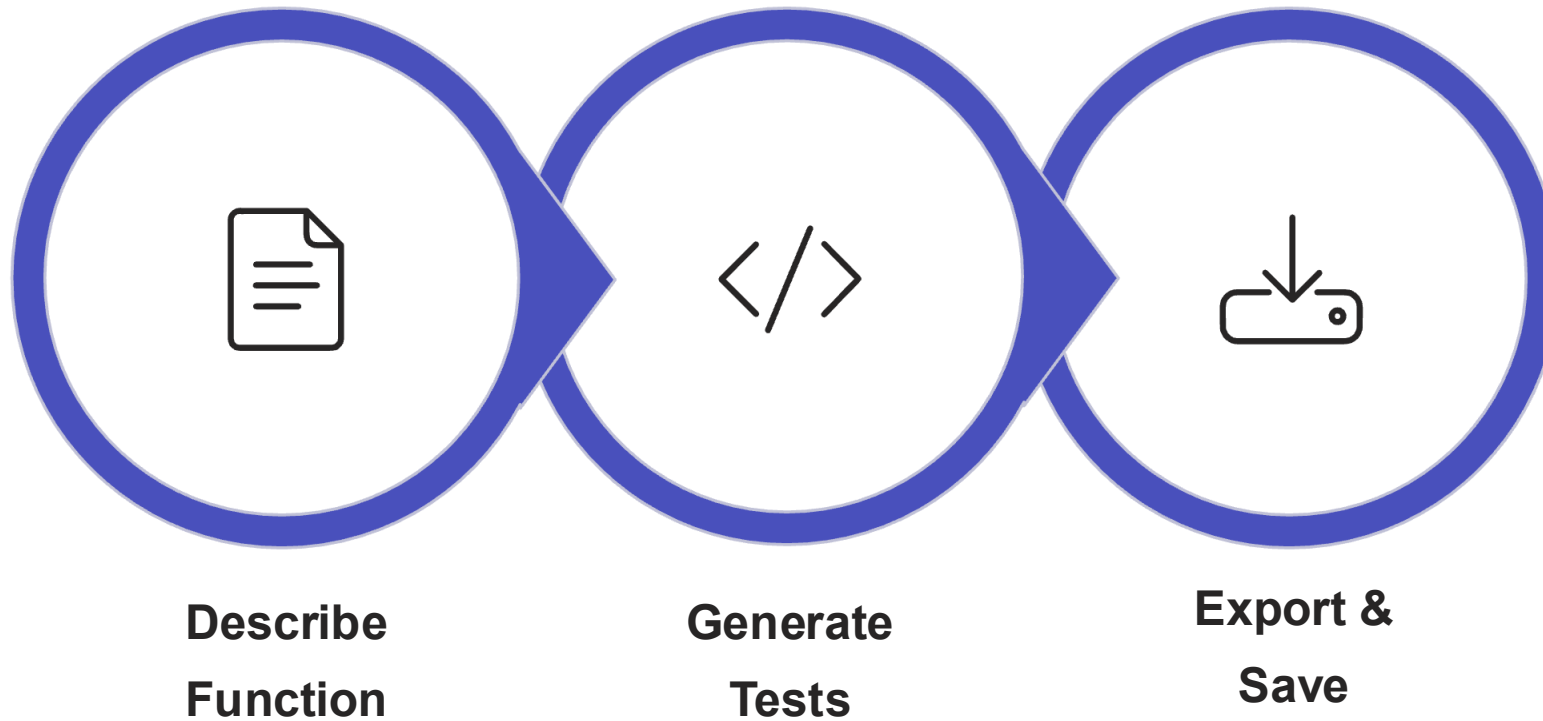
Inconsistent

No standard format across teams



Proposed Solution

A structured web tool – not a chatbot. The developer fills a form; the AI operates invisibly behind it.



No chat interface, no copy-pasting. Form input → structured data table → one-click .xlsx export

Target Users

- **Software Developers: Pain Point** – Manual Excel test case creation before every function slow
- **QA Engineer: Pain Point** – Inconsistent test case formats across developers
- **Tech Lead: Pain Point** – No standard documentation format within the team

Out of Scope:

- End users / citizens
- Project managers without coding background
- Non-TDD teams: this tool is specifically for developers writing pre-code test documentation

Six Core Features

AI Generation

Gemini API returns strict JSON – 6+ test cases per function across happy path, boundary, negative, and edge categories

Inline Editing

Any generated cell is editable before export

Excel Export

One-click .xlsx via ExcelJS with bold headers and alternating row colours

Auto-Persistence

Every session saved to Supabase PostgreSQL automatically

History Library

Search, filter, reload, and re-download any past session

Structured Tool

AI is invisible – no conversational UI, pure form-to-table design

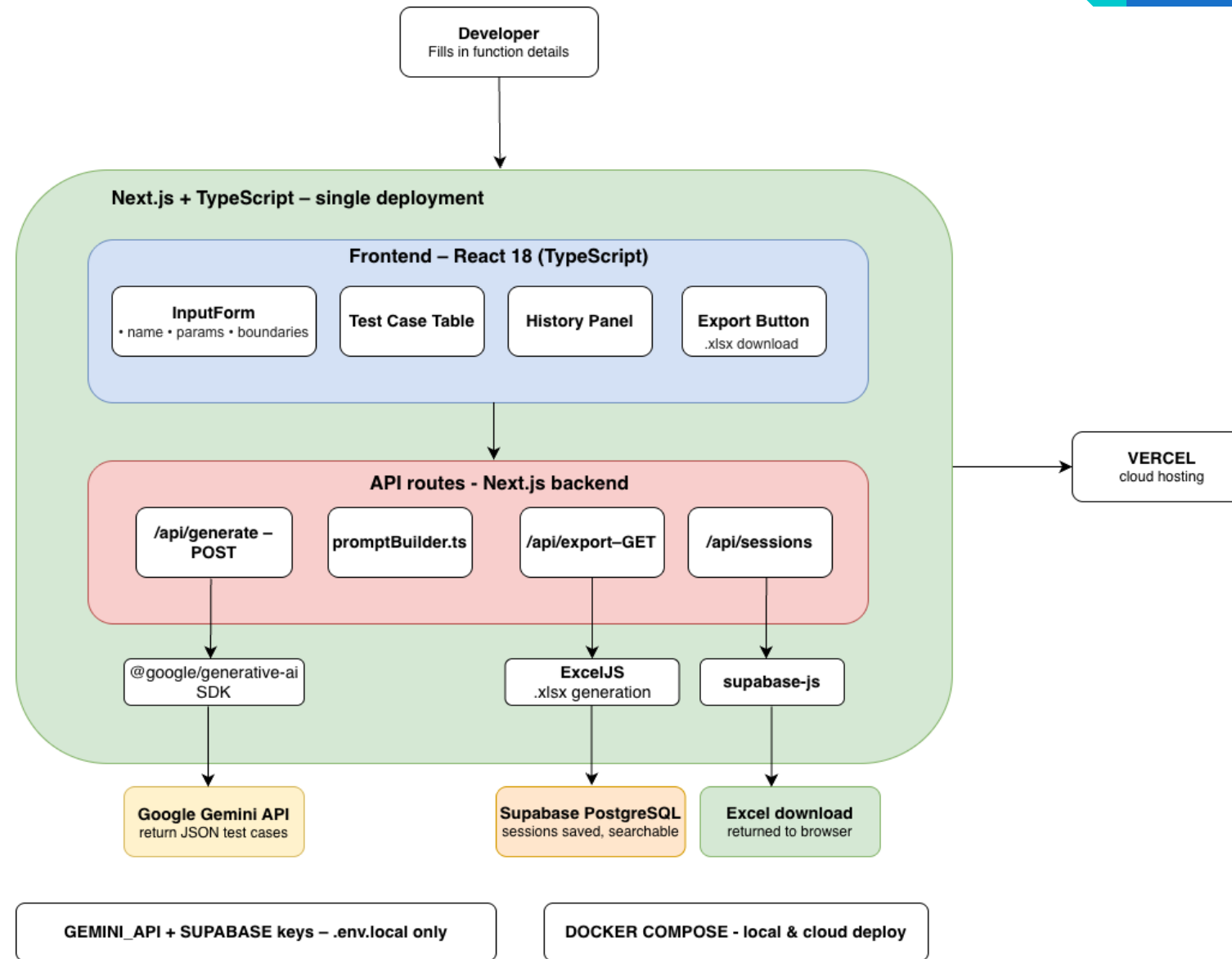
Technology Stack

Technology	Role	Rationale
 Next.js 14	Full-stack framework	Single codebase — UI + API routes; no separate server
 TypeScript 5	Static type system	Shared interfaces across all layers; compile-time safety
 React 18	Frontend UI Library	Input Forms, History, Export Functionality
 Google Gemini API	AI structured output	Strict JSON-only responses; no conversational UI
 Supabase	Managed PostgreSQL	Persistent session storage; free 500 MB; searchable history
 ExcelJS 4	Excel generation	.xlsx export with bold headers and alternating row colors
 Docker Compose	Containerization	one-command local deployment
 Vercel	Cloud Hosting	GitHub integration; env vars injected

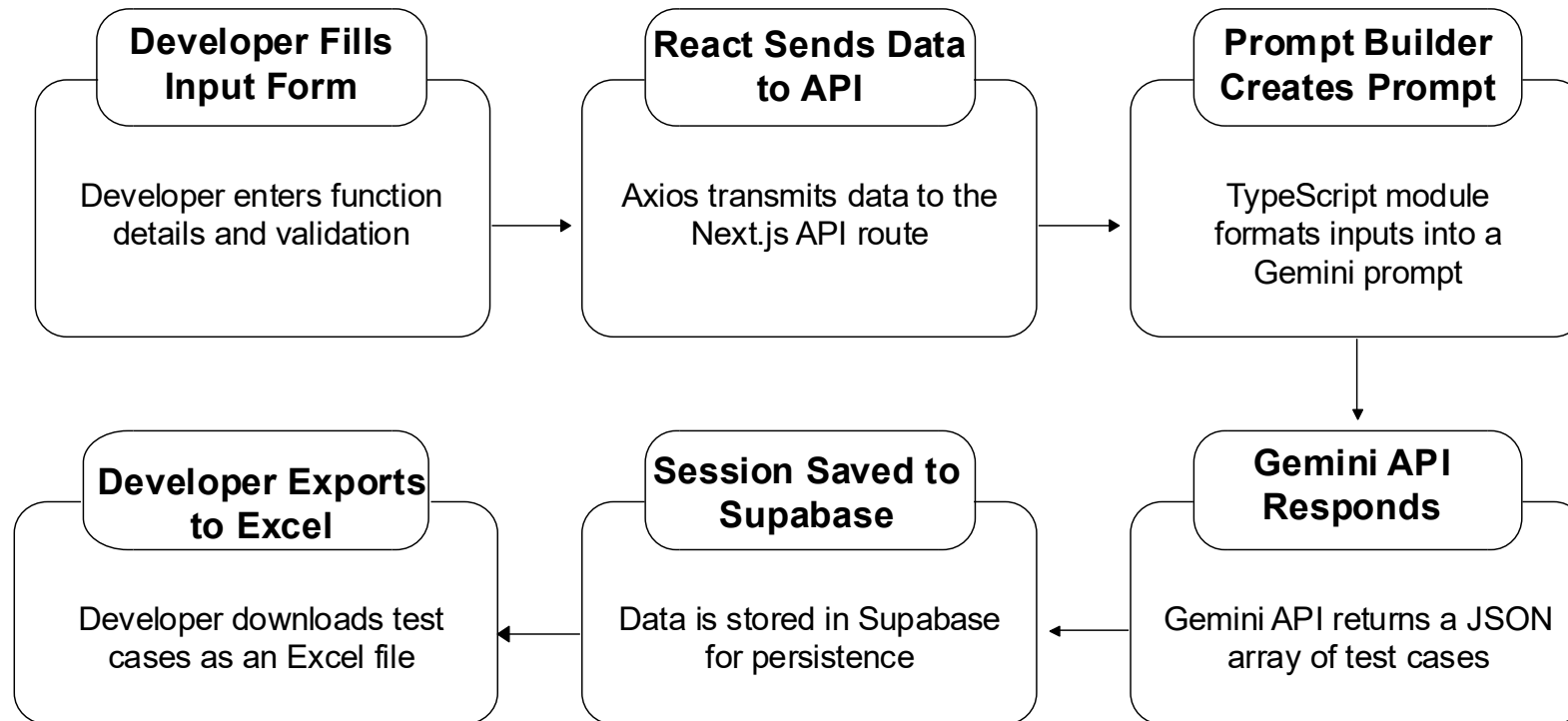
Database Design – Supabase sessions table

Column	Type	Constraint	Purpose
id	UUID	PRIMARY KEY	Auto-generated – gen_random_uuid()
function_name	TEXT	NOT NULL	Searchable – displayed in History Library
created_at	TIMESTAMP	DEFAULT NOW ()	Sort newest-first in session list
form_inputs	JSONB	NOT NULL	Full form data – repopulates InputForm on reload
test_cases	JSONB	NOT NULL	Test case array – repopulates table; used for re-export

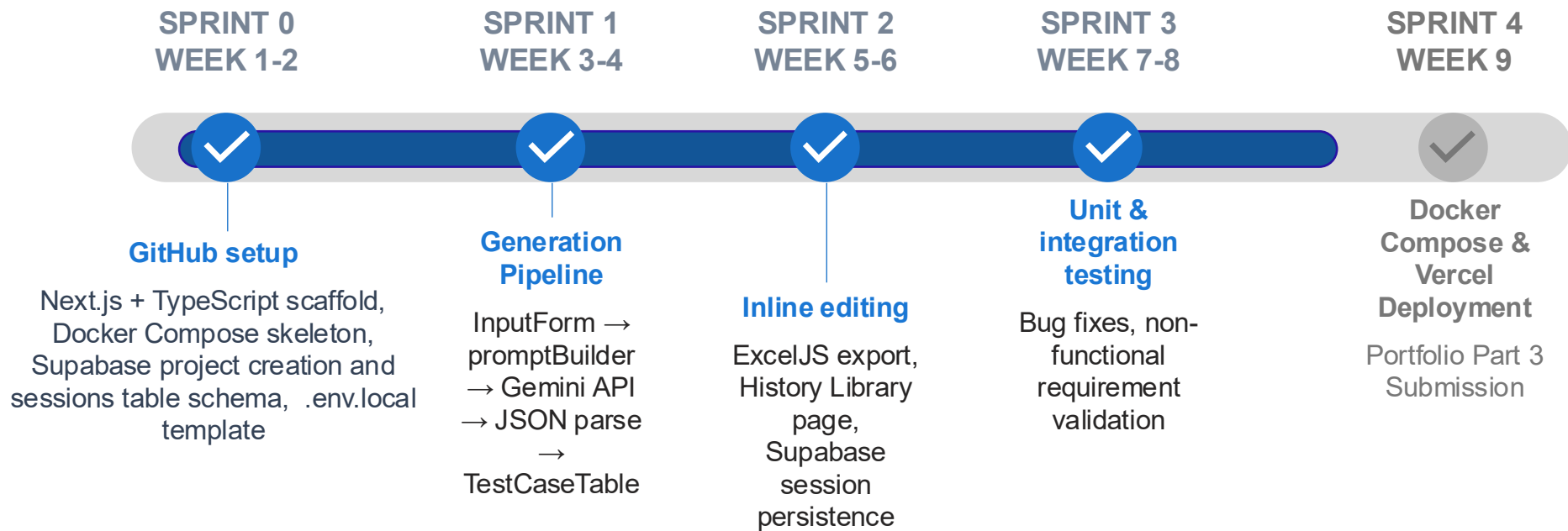
Architecture – System Component Overview



Architecture – Data Flow



Agile/Scrum Sprint Plan



Functional Requirements

- **FR-1:** System generates ≥ 6 categorized test cases per function submission
- **FR-2:** All sessions persisted to Supabase within 2 s of generation
- **FR-3:** History Library supports search by function name and filter by test category
- **FR-4:** .xlsx export produced with bold headers and alternating row colors

Non – Functional Requirements

- **NFR-1 Performance:** Test case generation completed end-to-end
- **NFR-2 Availability:** $\geq 99.5\%$ uptime via Vercel + Supabase free tier
- **NFR-3 Reliability:** Session data persisted reliably to Supabase
- **NFR-4 Maintainability:** All source files commented; TypeScript strict mode enforced
- **NFR-5 Scalability:** Backend handles concurrent requests

Risk Register and Mitigation

ID	Risk	Impact	Mitigation Strategy
R-01	Gemini API returns malformed JSON	High	promptBuilder.ts enforces strict JSON-only contract; fallback error boundary in UI
R-02	API rate limit hit (15 req/min)	Low	Exponential retry logic in /api/generate; user-facing error message with retry prompt
R-03	Supabase connection fails on Vercel	Medium	App degrades gracefully – generation still works; save fails silently with user notification
R-04	Excel file corrupt on download	Medium	Content-Type header enforced
R-05	Docker environment drift, Works locally, fails in CI/CD	Medium	Pin all image versions in docker-compose.yml; test container build in GitHub Actions

References

Academic Papers

- Karac, I., & Turhan, B. (2020). Why research on test-driven development is inconclusive? Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), 1–11.
<https://doi.org/10.1145/3382494.3410687>
- Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., & Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. IEEE Transactions on Software Engineering, 50(4), 911–936. <https://doi.org/10.1109/TSE.2024.3368208>
- Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering, 50(1), 85–105. <https://doi.org/10.1109/TSE.2023.3334955>
- Tosun, A., Ahmed, M., Turhan, B., & Juristo, N. (2018). On the effectiveness of unit tests in test-driven development. Proceedings of the 2018 International Conference on Software and System Process (ICSSP), 113–122.
<https://doi.org/10.1145/3202710.3203153>
- Sheta, S. V. (2023). The role of test-driven development in enhancing software reliability and maintainability. Journal of Software Engineering (JSE), 1(1), 13–21. <https://ssrn.com/abstract=5034145>

Books & Technical Documentation

- Beck, K. (2002). Test-driven development: By example. Addison-Wesley Professional.
- Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum — the rules of the game. Scrum.org. <https://scrumguides.org>
- Vercel Inc. (2024). Next.js 14 documentation. Next.js. <https://nextjs.org/docs>
- Google LLC. (2024). Gemini API documentation for Node.js. Google AI for Developers. <https://ai.google.dev/gemini-api/docs>
- Supabase Inc. (2024). Supabase documentation: PostgreSQL database and JavaScript client SDK. Supabase. <https://supabase.com/docs>
- ExcelJS Contributors. (2024). ExcelJS: Excel workbook manager. GitHub. <https://github.com/exceljs/exceljs>
- Docker Inc. (2024). Docker Compose documentation. Docker Docs. <https://docs.docker.com/compose/>



Thank You

Raksha Rajkumar Kademani

Matriculation No.: 4252827

Email Id: raksha-rajkumar.kademani@iu-study.org